Experiment 5

Analysis of Data Communication in a Web Application Using Browser Developer Tools

1. Aim

To inspect the requests and responses exchanged between a browser and a web application using built-in developer tools.

2. Objectives

- Open and use the browser Developer Tools Network panel.
- Identify request methods, status codes, headers, payloads, and response data.
- Understand the basic request-response cycle and the role of HTTP or HTTPS.
- Recognize common performance and communication problems from network evidence.

3. Requirements

A modern browser such as Chrome, Edge, or Firefox and an internet connection.

4. Theory

When a user opens a web page, the browser may request HTML, CSS, JavaScript, images, fonts, and data from APIs. The Network panel records these exchanges. A request commonly includes a URL, method, headers, and sometimes a payload. The server returns a status code, headers, and response content. Status codes such as 200, 301, 404, and 500 indicate different outcomes. Timing information can reveal slow server responses or large resources. HTTPS encrypts data in transit, although developer tools still show the browser's decoded request and response details where permitted.

5. Procedure

1. Open a sample website and launch Developer Tools using the browser menu or F12.
2. Select the Network tab and enable Preserve log if navigation needs to be recorded.
3. Reload the page to capture the requests made during loading.
4. Select a document, image, script, or Fetch/XHR request.
5. Inspect the Headers, Payload, Preview/Response, and Timing sections.
6. Record several request URLs, methods, status codes, and approximate loading times.
7. Reload with cache disabled and compare the results if you want to observe caching effects.

6. Case Study and Analysis

A small online bookstore reports that its search results sometimes appear slowly. A developer opens the Network panel and searches for a book. The HTML document loads quickly, but the Fetch/XHR request that retrieves search results takes much longer. The request URL and query parameters confirm that the browser is asking the correct endpoint. The response returns status 200, so the request technically succeeds, but the

timing breakdown shows that most of the delay occurs while waiting for the server. The developer can now investigate the API, database query, or server load instead of unnecessarily rewriting the page layout. In a second test, a product image returns 404 because its path is incorrect. This is a different problem: the request reaches a server, but the resource is not found. The Network panel therefore helps distinguish a slow successful request from a failed resource request. When inspecting real applications, sensitive tokens, personal data, and private payloads should not be copied into reports or shared publicly.

7. Result / Discussion

The Network panel exposed the resources requested by the browser, their status codes, headers, response details, and timing. These observations made it possible to identify likely sources of delay or failure.

8. Conclusion

Browser Developer Tools are a practical first step in diagnosing web communication issues. They help developers use evidence from actual requests rather than guessing at the cause.

9. Precautions / Best Practices

- Use fictional or non-sensitive test data during practice.
- Verify settings and results before using the application or configuration in a real environment.
- Record observations clearly and follow the platform's current documentation, security, and access-control guidance.